

Protocol-Governed Computing: Business Software That Runs on a Domain-Neutral Substrate

Short title: Business Software on a Domain-Neutral Substrate

Bhash Ganti

Independent Researcher

bachipeachy@gmail.com · ORCID 0009-0007-3810-6520

Abstract

Operating systems admit a new process, virtual machines new bytecode, and database engines a new query without being rebuilt for it. This paper asks whether governed business domains can likewise arrive as artifacts that a shared substrate admits and executes without acquiring domain-specific behavior.

We define two testable conditions: C1, admission without substrate modification, and C2, no domain-specific behavior in the substrate. In a protocol-governed computing composition, four heterogeneous domains are compiled into sealed snapshots and executed by one traversal engine. Inspection shows that 22 domain-specific capability implementations are namespaced to their domains, no domain vocabulary participates in an execution decision, and an injected cross-domain reference is refused before an artifact is produced. The executable substrate changed at each recorded admission, but each change occurred at a shared contract boundary and introduced no domain-specific behavior. Thus literal C1 was not demonstrated, while C1'—admission without domain-specific substrate modification—was.

Governed content reproduced byte-for-byte within the tested environment; renderer-dependent images prevented complete snapshot identity from reproducing across environments. The contribution is an explicit two-condition test for substrate independence at the business-domain layer, a realization demonstrating C1' and supporting C2 within the evaluated scope, and measurements that identify the boundaries of that result.

Keywords: software architecture; substrate independence; domain neutrality; declarative execution; sealed artifacts; capability-based systems; business process execution; reproducible builds

1. The problem

Why does a new business domain still require changes to the system that runs it, when other forms of software have long been admitted without changing their substrate?

Software has solved this problem before.

A new application runs on an operating system nobody rebuilt for it. New bytecode runs on a virtual machine that has never heard of it. A new query runs on a database engine that was compiled years earlier. In each case the substrate stays fixed. The new work arrives as an artifact the substrate knows how to consume, and the substrate consumes it.

A business domain here is a bounded area of business behavior with its own vocabulary, operations, rules and state — order handling for an e-commerce seller, loan origination for a lender, catalog management for a library. It is governed when those rules and operations are declared rather than left implicit in the code that carries them.

Governed business domains are commonly implemented through a combination of domain code, configuration, deployment material and runtime services. Admitting a new domain can therefore require changing the executable composition that already carries other domains. The domain does not arrive as an artifact. It arrives as a change to the system that runs it.

This paper asks whether that coupling can be removed instead. Can a governed business domain arrive as an executable artifact that a shared substrate admits and executes without acquiring behavior specific to that domain? The question is narrower than general software modularity: it concerns the boundary between a governed domain artifact and the executable substrate that consumes it.

The paper does not propose another business-process framework. It examines how far two things can hold together for governed business domains: admitting a new domain without modifying the substrate, and keeping behavior particular to that domain out of the substrate. Section 1.2 states both as named conditions, Section 5 gives the mechanism, and Section 6 reports which of them the realization meets. The contribution is architectural and evidentiary — a defined property, a realization of that property within a stated scope, and measurements that test the realization's domain neutrality.

1.1 What existing substrates already achieve

A natural objection is that substrates admitting new work without modification are everywhere, so the problem may already be solved.

It is worth separating the substrates that solved it from those that appear to.

Three substrates achieve the property cleanly, each for its own unit of work.

- An **operating system** admits a new process. It schedules and arbitrates resources. It does not decide what the process computes.
- A **virtual machine** admits new bytecode. It resolves classes and compiles methods at run time. Those decisions affect execution mechanism, not the program's domain behavior.
- A **database engine** admits a new query. It plans execution. The plan is chosen for cost, while the query's specified result remains determined by its semantics.

In each case the substrate is ordinarily specified as an execution mechanism rather than as an implementation of the admitted application's domain behavior. These examples establish architectural precedent for the separation. They do not establish that business domains are equivalent units of work, or that the separation transfers without additional constraints. The remainder of the paper makes those constraints explicit as two named conditions, defined next.

Three further substrates admit domain-shaped work and still contribute domain decisions.

- A **container orchestrator** admits a new workload without modification [1]. It then chooses the path from current state to declared state. Restart policy, backoff, eviction and rollout order are decisions the workload did not carry.
- A **workflow or process engine** admits a new process definition without modification [2, 3]. It then evaluates gateway expressions, runs script tasks and binds handlers at run time. Those are domain decisions made by the engine.
- A **policy engine** admits new policy without modification [4]. It then evaluates that policy at the moment of a request. The decision is produced by the engine, not carried entirely by the artifact.

Each of these admits new work unmodified. None of them stops there.

The criterion. The distinction is best made by asking where a domain-visible decision is *determined*, not where it is written:

A substrate contributes domain-specific behavior when, for some admitted artifact, the artifact does not determine a domain-visible outcome and the substrate resolves it.

Mechanism decisions do not count. A database engine may choose an index, and a runtime may choose an evaluation order, without changing the specified result. The criterion concerns domain-visible outcomes.

The criterion also concerns determination rather than authorship. A workflow author may write a gateway expression, but if the runtime evaluates that expression and determines the resulting path, the runtime remains a source of that decision.

The classifications above are therefore characteristic rather than absolute. Systems move along this axis as their execution model changes: an orchestrator with no admission webhooks contributes less than one with them. The purpose of the criterion is to make the comparison explicit and testable. Section 8 places the same substrates against C1 and C2 individually.

1.2 The property, defined

The distinction above needs a test, or the objection recurs at every turn.

First the unit of admission. Throughout this paper a domain is admitted as an **executable domain artifact**: a sealed representation of domain definitions, state and references that a fixed runtime can validate and execute. The architecture that produces and consumes such artifacts is **protocol-governed computing (PGC)** [5, 6], and the realization studied here is a PGC composition. PGC

names that approach and nothing broader; it is not used as a synonym for substrate independence.

Substrate independence then requires two conditions, and both must hold at once.

- **C1 — Admission without substrate modification.** The substrate admits a new domain without the substrate itself changing.
- **C2 — No domain-specific behavior in the substrate.** The substrate contributes no behavior particular to the domain it admits.

Operating systems, virtual machines and database engines satisfy both — for processes, bytecode and queries. Orchestrators, workflow engines and policy engines satisfy C1 while retaining domain-specific decisions. The question this paper investigates is whether both conditions can be achieved for governed business domains.

The rest of this paper keeps the two conditions apart and measures each one separately. Keeping them separate prevents C1 from being mistaken for the full property: many existing substrates admit new work without modification while still contributing decisions the admitted artifact does not carry.

Two qualifications belong here rather than in the results, because they change what the reader should expect the paper to establish.

Substrate in C2 means the shared executable engine that every admitted domain runs on, defined precisely as part (a) in §5.2. It does not mean the whole running composition. The realization studied here executes domain-specific code that is namespaced to its domain and carried outside

both the substrate and the sealed artifact, and C2 makes no claim about that code beyond where it sits.

C1 as stated above is the strongest form of substrate independence. The realization does not establish it: Section 6.1 reports that the executable substrate changed during the recorded admissions. The experiment therefore also evaluates a weaker but useful property:

- **C1' — Admission without domain-specific substrate modification.** A new domain may require a change at a shared contract boundary, provided that the change introduces no behavior particular to that domain.

C1' does not replace C1. It captures a distinct architectural question: whether domain growth changes the substrate because the domain requires new behavior, or because the shared interface between artifacts and the substrate is being evolved. Section 6 measures that distinction directly. C1 remains the stronger target; C1' is the property demonstrated by the present realization.

1.3 Relation to concurrent submissions by the same author

Four companion papers are under review elsewhere, and all five rest on the same architecture and the same reference realization. The shared basis is unavoidable and is disclosed here so that overlap can be checked rather than assumed. What differs is the question each paper asks, and each question appears in no other paper as a question.

Paper	Question	Method
[7]	Why does agent-mediated development break authorization?	position, with three tests
[8]	What follows architecturally from adopting the governed properties together?	derivation from stated premises

Paper	Question	Method
[9]	Can an agent perform the engineering work without gaining authority over the result?	empirical study of one transformation
[10]	What is the lifecycle architecture, and how is conformance to it specified and checked?	specification and conformance assessment
this paper	Can a shared substrate admit and execute a governed business domain without acquiring its behavior?	architectural case study with targeted falsification tests

This paper asks a substrate question the others do not ask. Where their results are needed here they enter as cited premise or as positioning, never as restated contribution: [5] and [6] supply what a protocol-governed composition is, [11] supplies the conformance vocabulary, and [9] is cited in Section 2 and in the disclosure, for the arrangement under which parts of the realization were constructed. No text, argument, data or evidence from any of the four is reused here. The measurements in Section 6 are taken from the deposited composition [12] by the method of §5.6 and appear in no other paper.

2. What forces the question now

Machine-generated software makes the separation more consequential [9]. When new domain behavior can be produced rapidly, changing the shared substrate can become the slower and broader part of the process.

The issue is not generation speed itself. It is the change surface that generation exposes. If adding one domain requires modifying the substrate, the new domain can affect the executable composition shared by existing domains. If the domain instead arrives as a bounded artifact, its change surface can remain local to that artifact.

This paper therefore treats substrate independence as a structural property rather than as a performance claim. The question is whether domain growth can be represented primarily as the admission of a new artifact rather than as the modification of the mechanism that executes existing artifacts.

The change surface matters most when admission is itself automated. If admitting a domain requires changing the substrate, then automating admission requires automating a change to the substrate. The automation must then author part of the system that every other domain already runs on. A pipeline intended to add one domain acquires the ability to modify the thing carrying all of them.

Two consequences follow wherever this binding holds.

First, a change with no relation to existing domains can still reach them. The substrate is shared, and re-deriving it for one domain produces a new substrate for all. Existing workloads therefore become part of the change surface for work that has nothing to do with them.

Second, guarantees established for the existing composition must be re-established when the substrate changes. The previous composition and the new composition are not the same executable substrate, so properties that depended on that substrate cannot simply be carried forward as facts about the successor.

These are structural consequences of a domain-derived substrate, not measured findings of this paper. A reader who disputes either characterization can do so without changing the results reported later. The empirical question begins when the paper asks whether the binding can be removed instead: can a new domain arrive as a sealed artifact without placing domain behavior into the substrate that runs it?

3. From a domain-derived substrate to a domain-bound substrate

3.1 The status of this section

What follows is the paper's diagnosis. It is argued, not measured. It explains why the asymmetry in Section 1 persists and motivates the mechanism described in Section 5.

It is not a result. The results are in Sections 4 and 6, and they do not depend on this diagnosis. A reader who rejects the explanation can still check C1 and C2 against the measurements.

The object of study is the runtime and artifact relationship, not the full organizational lifecycle of a business domain. The paper evaluates admission, execution, persistence and refusal behavior at the boundary between a fixed substrate and a sealed snapshot. It does not evaluate whether the admitted domains are economically useful, whether their policies are substantively correct, or whether the architecture is superior on performance or cost.

3.2 Where domain knowledge sits

Conventional substrates are not badly built. They are **derived from** the domain rather than **bound to** it.

In a conventional service-oriented deployment, domain knowledge is commonly distributed across executable and declarative components, and across the deployment material that composes them [13]. Services encode domain operations. Handlers encode domain events. Routing tables encode which operation reaches which implementation. Deployment descriptors encode which services exist and how they find one another. Configuration can add further domain-specific bindings and behavior.

The point is not that every conventional system has this exact arrangement. The point is that where these components form part of a shared executable composition, changing the domain can require changing the composition that carries multiple domains.

The consequence is that domain isolation depends on how these components are partitioned. When domain knowledge is embedded in a shared executable composition, a change to one domain can reach components that also carry other domains.

The problem is therefore structural rather than necessarily procedural. The more domain knowledge the shared substrate carries, the larger the change surface associated with adding or changing a domain.

3.3 The inversion

The alternative is to move domain knowledge out of the shared substrate.

A domain arrives as data: declarations, compiled structure, bindings and evidence. The substrate supplies the mechanisms needed to realize that structure — scheduling, persistence, error handling, serialization and capability enforcement — but it acquires no rule or branch particular to the domain.

The resulting invariant is **domain neutrality**. Domain neutrality does not require a small substrate. It requires that substrate growth be driven by shared execution concerns rather than by the business domains admitted to it.

Under this arrangement, admitting a domain produces a new sealed artifact rather than a new domain-specific substrate. Change therefore becomes a transition between governed states:

snapshot n to snapshot $n+1$, where the predecessor is named, the successor is derived from it, and the predecessor is not edited in place.

This is the architectural relationship the remainder of the paper examines: the substrate carries no domain, while the domain is carried by the artifact bound to it.

Section 4 reports what happened when that relationship was built and exercised. The mechanism follows in Section 5.

4. Evidence: scope and outcome

Before the mechanism is described, here is what was built and what it showed. Everything reported in this section and in Section 6 was read from the deposited composition [12] and from the specification it claims conformance to [11]; both are citable objects, not descriptions of them. Section 5 gives the mechanism, and Section 6 gives the detailed measurements, using the vocabulary defined there.

4.1 Breadth

Four governed domains were admitted to one composition:

- Decentralized identity and wallet management.
- AI agent governance and AI licensing.
- Library catalog management.
- A computational conformance workload.

They share no domain vocabulary, capability implementations or structural assumptions. The breadth is therefore heterogeneity rather than four variations on one domain shape.

Governed domain material is larger than the platform material that carries it. The domains are not small attachments to a large platform. The composition therefore tests whether one substrate can carry materially different domain artifacts rather than several instances of a common domain pattern.

How they were selected. They were not sampled from a population, and no claim of representativeness follows from them. They were chosen to differ along the axes on which a substrate could plausibly fail to stay neutral: the shape of the control flow, the persistence modes reached, the outcome codes a workflow must route on, whether an external boundary exists, and whether the domain needs computation the platform does not supply. A domain that exercised none of those would test admission but not neutrality.

What they exercise. The counts below are read from the composition by inspection, and they bound the generalization the paper is entitled to.

	Identity and wallet	AI governance and licensing	Library catalog	Conformance workload
Subdomains	2	2	1	1
Workflows	5	4	11	1
Compiled execution paths	38	43	123	6
Distinct outcome codes routed on	6	7	7	5
Declared stores	6	4	7	1

	Identity and wallet	AI governance and licensing	Library catalog	Conformance workload
Persistence modes reached	append-only, mutable, registry, clock	append-only, mutable, registry, clock	append-only, mutable, registry	mutable
Domain-declared transforms	1	3	4	2
External ingress declared	3	none	none	1

The four domains exercise different structural demands on the substrate. The catalog domain has an order of magnitude more compiled execution paths than the conformance workload. The identity domain is the only one exercising an external ingress boundary. The domains reach different persistence modes, and all four declare computation the platform surface does not supply — the case where domain neutrality is hardest to keep.

The selection therefore provides variation in the dimensions most relevant to C2: topology, routing outcomes, persistence, external ingress and domain-specific computation. It does not establish representativeness of business software as a whole.

What they do not exercise, and it matters. Four domains do not cover the space of business software, and the gaps are specific rather than rhetorical.

- **No external service interaction.** The governed side-effect surface has no network capability. Domains whose behavior depends on calling another system — payment authorization, credit decisions, most integration-heavy work — are not represented, and nothing here shows the property survives one.

- **Two of the six side effects are untouched by business domains.** Persistence is file-backed JSON, append-only streams and registries. No relational, transactional or distributed store is exercised, so behavior under concurrent writers or partial failure is untested.
- **No long-running or time-driven workflows.** Every workflow completes within one traversal. There are no timers, no waits, no compensation or rollback across steps, and no multi-party coordination.
- **One subdomain shape dominates.** All four are record-oriented: validate, claim an identity, write a record, append an occurrence. A domain organized around computation over a large state space, or around continuous rather than discrete events, would exercise a different part of the graph model.

A reader should read the results of Section 6 as bounded by this table rather than by the phrase *four heterogeneous domains*.

4.2 The workload counts as a domain

The repository names one of the four a conformance workload rather than a business domain. That name is a namespace convention, not an architectural distinction.

The platform admits the workload through the same path as the others. It seals it into the same kind of artifact and executes it under the same obligations, artifact kinds and refusal behavior — the obligations the specification states for any admitted domain [11].

The substrate gives the workload and the business domains the same treatment. That matters to C2. If the substrate had to recognize the workload as a special case, it would be contributing

behavior based on the nature of the admitted domain rather than merely realizing the artifact it receives.

4.3 Outcome

In the evaluated composition, one executable substrate carried all four selected domains.

No admission introduced domain-specific behavior into that substrate. Domain-specific implementations remained outside it, and the inspected substrate contained no domain-specific vocabulary, routing or handler implementation for the admitted domains.

The executable substrate did change during the recorded admissions. Those changes occurred at the shared contract boundary between the runtime and the snapshot. None added behavior for the domain being admitted. One change guarded the loading of handler code that the substrate does not own, so that a missing module produces a declared refusal rather than a crash.

That gives two results rather than one. Literal substrate immutability, C1, was not demonstrated. Domain-neutral substrate evolution, C1', was demonstrated for the recorded admissions. C2 is supported by the implementation inventory and substrate inspection reported in Section 6. Section 6 counts the churn, identifies the affected modules, and classifies each change as contract-boundary or domain-specific. Any domain-specific change would refute C2.

4.4 What this evidence is not

The evidence comes from one practitioner-built composition holding four heterogeneous domains. It is sufficient to test whether the same admission path, artifact obligations and refusal behavior apply across those domains. It is not sufficient to establish comparative performance, reliability, cost or general prevalence.

No conventional system was built alongside for comparison. One practitioner built the realization, and that practitioner also designed the architecture.

Section 7 states the limits in full, once the mechanism has been given.

5. The mechanism

Sections 1 to 4 defined the property, argued why it is worth having, and reported what was built. This section gives the mechanism that produced those observations: what crosses the substrate boundary, in what form, and what the substrate does with it.

The section is descriptive rather than evaluative. It states how the realization works and defines the vocabulary Section 6 measures against. Nothing here is offered as evidence; the measurements are in Section 6, and a reader may check them against the definitions given here without accepting the diagnosis in Section 3.

Three registers, kept apart. A description of an architecture and a report about a realization are easy to interleave and hard to separate afterwards, and the difference decides what a reader is entitled to conclude. Every load-bearing statement in this section belongs to one of three registers, and the register is marked wherever the statement is made.

- **Required** — an obligation of the architecture, specified in the normative family [11] and cited here rather than re-argued. A realization that violates it is non-conforming; that no realization violates it is not thereby shown.
- **Enforced** — an invariant a compiler checks, refusing to build when it fails. Each carries a named check, so the claim is falsifiable by running the compiler rather than by trusting the text.

- **Observed** — a fact about the composition studied here, established by inspection or measurement and reported in Section 6. It holds for this realization and carries no general claim.

The distinction matters most where the three are easy to confuse. *Execution determines no behavior* is Required. *A side effect outside the declared surface does not compile* is Enforced, by a named invariant. *The runtime contains no domain vocabulary* is Observed, by inspecting it. The first two say what the architecture obliges and what the toolchain refuses; only the third is a finding, and only the third could have come out otherwise.

Statement	Register	What stands behind it
Behavior is determined before execution; execution adds none	Required	the normative family [11]
A refused change produces no snapshot and names the rule that refused it	Required	the normative family [11]
The platform side-effect surface is exactly six	Enforced	INVARIANT_CS_SURFACE_CLOSED_V1, a compiler assertion naming its six members
The platform transform surface is exactly twelve	Enforced	INVARIANT_CT_SURFACE_CLOSED_V1, the same mechanism

Statement	Register	What stands behind it
The runtime synthesizes no route	Enforced	INVARIANT_NO_RUNTIME_TOPOLOGY_SYNTHESIS_V0
Routing is complete in the compiled graph, so no fallback is needed	Enforced	INVARIANT_TOPOLOGY_ROUTING_COMPLETE_V0
The compiled topology is immutable after compilation	Enforced	INVARIANT_TOPOLOGY_IMMUTABLE_AFTER_COMPILATION_V0
A domain may not reference an unauthorized namespace	Enforced	INVARIANT_FQDN_NAMESPACE_AUTHORIZED_V0
The runtime holds no domain vocabulary, routing table or handler	Observed	inspection of part (a); Section 6
Admission changed no artifact outside the domain's declared paths	Observed	Section 6, reported as a distribution
This workflow carries exactly eight execution paths	Observed	the compiled graph, read by inspection

The composition carries 92 such invariants, of which the majority are checked as compiler assertions with an immediate-failure response. The table names only those the argument of this section leans on.

Protocol-governed computing (PGC) is a model of computation rather than a program. All behavior originates from a compiled, governed artifact, and a fixed engine executes that artifact with no knowledge of the domain it encodes. Behavior is determined and validated before execution; execution is a traversal that produces an observable trace. The architecture is specified independently of any one compiler, runtime or language — in the way SQL is specified independently of any one database engine, and bytecode independently of any one virtual machine. Those are the same two substrates Section 1.1 used to establish precedent, and the analogy is deliberate: the claim under test is that a business domain can stand in the same relation to its substrate that a query stands in to a database.

5.1 Three functions, and the one this paper examines

A governed system's life is three functions over one architecture.

$$(B_n, P) \xrightarrow{\mathcal{T}} B_{n+1} \xrightarrow{\mathcal{C}} S_{n+1} \xrightarrow{\Phi} (R, T)$$

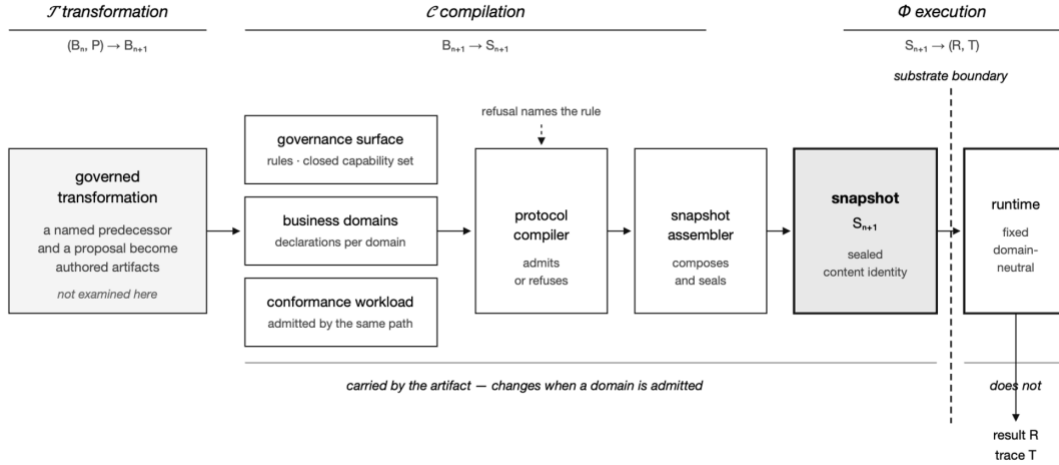
- **$\mathcal{T}$, transformation.** A named predecessor baseline B_n and a proposal P become a successor baseline B_{n+1} . The predecessor is named, not edited.
- **$\mathcal{C}$, compilation.** The successor baseline is admitted, composed and sealed into a snapshot S_{n+1} .
- **Φ , execution.** The snapshot produces a result R and a trace T , and produces nothing else.

Each function may refuse. A proposal with no legitimate successor stops the pipeline, which reports which phase refused it and under which rule; it does not yield a weaker baseline instead. A refusal is an artifact of the process rather than the absence of one. Because a baseline becomes a snapshot only at $\mathcal{C}$, a change refused anywhere before that point never reaches a running system.

What this paper examines. The subject here is Φ , together with the snapshot as the product of $\mathcal{T}$ and $\mathcal{C}$. The transformation function is a governed lifecycle in its own right, carried by two further compilers — one taking a stated problem to a design mandate, the other taking that mandate to authored artifacts — and it is treated in companion papers [6, 10]. This paper cites that treatment rather than re-arguing it, and takes the snapshot as given: a sealed artifact of known provenance, arriving at the substrate boundary.

That restriction matters to the argument. C1 and C2 are properties of the boundary between a sealed artifact and a fixed substrate. They can be stated, measured and falsified without any claim about how the artifact was authored. A reader who doubts the transformation account can still check both conditions.

Figure 1 places the three functions over the realization that carries them, and marks the boundary the rest of this paper measures at.



Two kinds of authored material. Left of the boundary, the material divides in a way that carries the C2 argument. A **governance surface** declares the rules a composition is governed by and the closed set of operations any domain is permitted to perform. A **business domain** declares behavior in terms of that surface. The surface defines the alphabet; a domain writes sentences in it and may not invent a letter. A domain that uses an undeclared operation does not fail at run time — it does not compile.

The conformance workload of Section 4.2 sits in the same position as the three business domains. The compiler applies the same admission rules to it, the assembler seals it into the same kind of artifact, and the runtime traverses it by the same path.

5.2 The boundary, defined

C1 and C2 are claims about a substrate, so neither can be evaluated until the substrate is bounded. A running composition has five parts, and the conditions quantify over some of them and not others.

Part	What it is	Where it lives
(a) Executable substrate	the traversal engine: scheduler, dispatcher, loader, evidence and supporting modules	a fixed package, identified by content identity
(b) Sealed snapshot	declarations, execution graphs, indexes, vocabulary, bindings, admission evidence, manifest	content-identified; carries no executable code
(c) Capability implementations	the modules that realize declared transforms and side effects	named by artifacts in (b), resolved from outside it at load time
(d) Stores and external state	the state governed operations read and write	declared in (b); their contents are external and mutable
(e) Deployment material	which snapshot is loaded, where stores live, where modules are imported from	environment and configuration; carries no behavior

Part (c) determines how far the identity claims in this section reach, so it is worth stating precisely. A binding records a module path and a callable name — nothing else. It carries no content digest, no version, no repository identity and no signature. A different module presented under the same name satisfies the binding, and neither the snapshot identity nor the substrate identity changes when it does. The sealed artifact therefore names the code it needs without constraining which code answers to that name, and the reader should carry that through every identity claim in the paper. The snapshot itself contains no executable code: it carries declarations, graphs and bindings, and the code that realizes a capability is loaded separately by name. Those implementations come in two kinds. **Platform capabilities** — the shared transforms and the six side effects — are carried with the governance surface and are common to every composition. **Domain capabilities** are namespaced to the domain that declares them and ship with it: the identity domain, for example, declares its own transform for deriving a wallet

address from key material. Domain computation therefore stays attached to the domain that declares it rather than accumulating in the shared substrate.

Determination and realization. The layout above separates two things that the phrase *the domain arrives as an artifact* runs together. The snapshot carries **behavioral determination**: what will happen, on which path, under which binding, to which store. Capability implementations carry **behavioral realization**: the code that performs it. A domain therefore arrives as a sealed governed artifact together with externally resolved implementation dependencies, and the paper uses that pairing throughout rather than treating the snapshot as a standalone software distribution.

The separation raises three distinct questions, and keeping them apart is what makes the results precise.

Where is behavior determined? The snapshot answers this. Declarations, graphs and bindings cross the boundary sealed and content-identified, so execution requires no additional domain knowledge.

Where is behavior performed? Capability implementations answer this. A domain-specific pure transform is behavior particular to a domain, executing inside the composition, in code the sealed artifact names but does not contain. C2 is therefore not the claim that no domain-specific code runs anywhere. It is the claim that no such behavior sits in (a) — that domain computation travels with the domain that declared it rather than accumulating in the substrate every domain shares.

Which implementation performed it? The current binding mechanism does not fully answer this, because it names a module and callable without fixing their content identity. The first question is

central to C2. The second is expected by the architecture. The third is an implementation boundary the present realization does not yet close.

That third boundary also bears on the C1 measurement. The runtime's content identity covers (a) alone and not (c), so the executable substrate can be byte-identical across an admission while the modules it loads have changed entirely. Section 5.7 therefore measures the identity of (a) and reports (c) separately, rather than folding the two into one claim of an unmodified system.

What would falsify C2 under these definitions is specific and checkable: a capability implementation that is particular to one domain and is carried with the substrate rather than with that domain, or any domain vocabulary, routing table or handler name appearing in (a). Section 6 looks for both.

5.3 Anatomy of a snapshot

The snapshot is the object the architecture turns on. Everything before it produces it; everything after it consumes it and may not change it. It is not one build output among others: it is the composition, at one moment, in a form that can be named, checked and executed.

The organizing question is not what a snapshot happens to contain. It is:

What information must cross the substrate boundary so that execution requires no additional domain knowledge?

That question turns the anatomy from documentation into a test. Each component earns its place by answering one execution question. A component that answers none does not belong in the snapshot. A question the runtime has to answer for itself is a failure of C2, because answering it would require the substrate to hold something particular to the domain. Section 5.2 gave the

answer this classification assumes: what crosses sealed is everything that determines behavior, while the code that performs it crosses by name.

Snapshot component	Execution question it answers
Declarations	What behavior is admitted?
Projections	What executable structure realizes it?
Capability binding map	Which permitted capability realizes each operation?
Indexes and vocabulary	What exists, under what name, and where?
Admission record	On what ground was each artifact admitted?
Manifest	What constitutes this snapshot?
Content identity	Which exact snapshot is being executed?

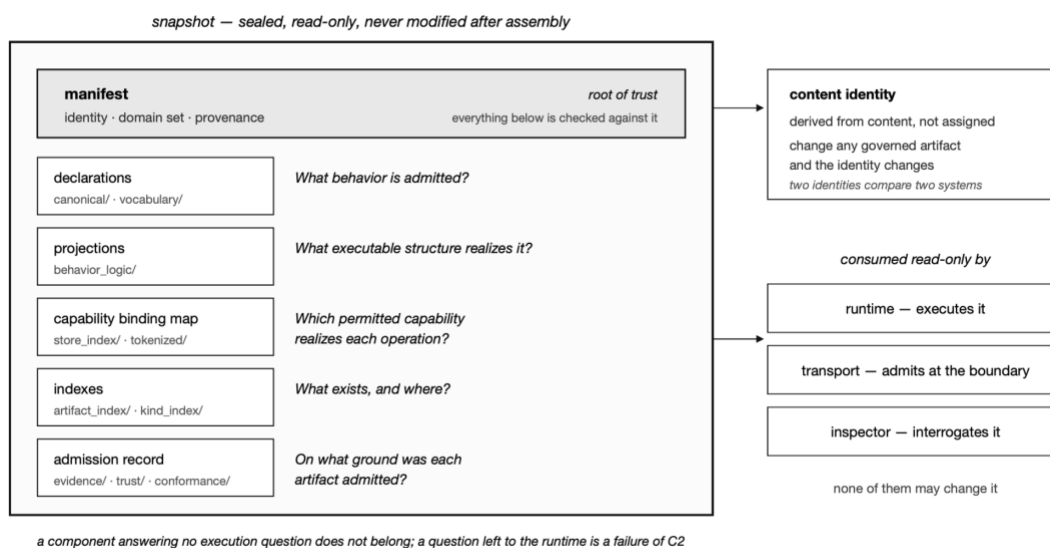
The realized layout follows that classification directly: canonical declarations and a vocabulary of every named concept; behavior logic holding the execution graphs; artifact, kind and store indexes; address-resolved forms; per-artifact admission evidence and the composition-conformance record; and a manifest at the root.

The indexes are not conveniences. They provide an operational basis for inspecting the admitted system without reconstructing it from source code, and the measurements in Section 6 are taken by interrogating them rather than by reading code.

Sealing and identity. A running system is normally a set of parts that happen to be deployed together, and answering *what exactly is running* means reconstructing it from versions, configurations and environments. A snapshot answers that question directly, because its identity is derived from its content rather than assigned to it. Change any governed artifact anywhere in

the composition and the identity changes. Two people comparing identities are comparing the systems, not their descriptions of them.

That property is what makes the rest of the architecture expressible. To pin a baseline is to name a snapshot. To ask whether two systems are the same is to compare two identities. To claim a build is reproducible is to rebuild it and get the same identity back — a claim §6.6 tests and partly falsifies. It is also what makes the admission experiment in Section 5.7 a measurement rather than a narration: step 3 compares two identities, and identity is defined before the comparison is made.



5.4 The domain-neutral runtime

The runtime reads a sealed snapshot and executes the workflows in it. What distinguishes it is what it does not contain.

It holds no rule, no policy and no branch that depends on what a workflow means. It does not discover behavior, interpret intent or carry business logic. Everything it will do was determined

at compile time, and execution is a traversal of what the snapshot already says. There is no path in the substrate by which a route absent from the compiled graph could be constructed.

This is not an application server, nor an orchestration engine that evaluates rules at request time, nor a framework a domain plugs behavior into. Behavior is not extended by writing runtime code. It is extended by declaring more protocol, compiling it, and sealing a new snapshot. That distinction is exactly the one Section 1.1 drew between substrates that satisfy both conditions and substrates that satisfy C1 alone: a workflow engine admits a new process definition unmodified and then evaluates gateway expressions itself, which is a domain decision made by the engine.

Six properties define the substrate's contribution, and each is stated so that it can be checked rather than asserted.

1. **No domain knowledge.** The substrate contains no vocabulary, routing table or handler implementation particular to an admitted domain.
2. **Execution adds no path.** Routing comes only from the compiled graph. The runtime computes no route the snapshot did not carry.
3. **No fallback.** A missing binding, capability or route is a declared refusal, never a default.
4. **All mutation is through declared capabilities.** There is no other write path into governed state.
5. **The snapshot is read once and never modified.**
6. **Every run emits a trace,** produced by the same traversal that produced the effects, so the record and the behavior cannot diverge.

Property 3 carries more weight than it first appears to. A fallback is how domain knowledge usually enters a substrate unobserved: the engine supplies a default, the default becomes part of

the system's effective semantics, and nothing declared it. Refusing instead keeps the substrate's contribution enumerable.

What the substrate is permitted to do. The operations available to any domain divide in two, and the asymmetry between them is structural rather than conventional. The model is the object-capability one [14, 15]. A **capability transform** is pure computation: inputs to outputs, no files, no network, no clock, no unseeded randomness, no global state. A **capability side effect** is a governed mutation — a change to something outside itself, such as a store or the clock. Every change a composition can make to anything is one of these.

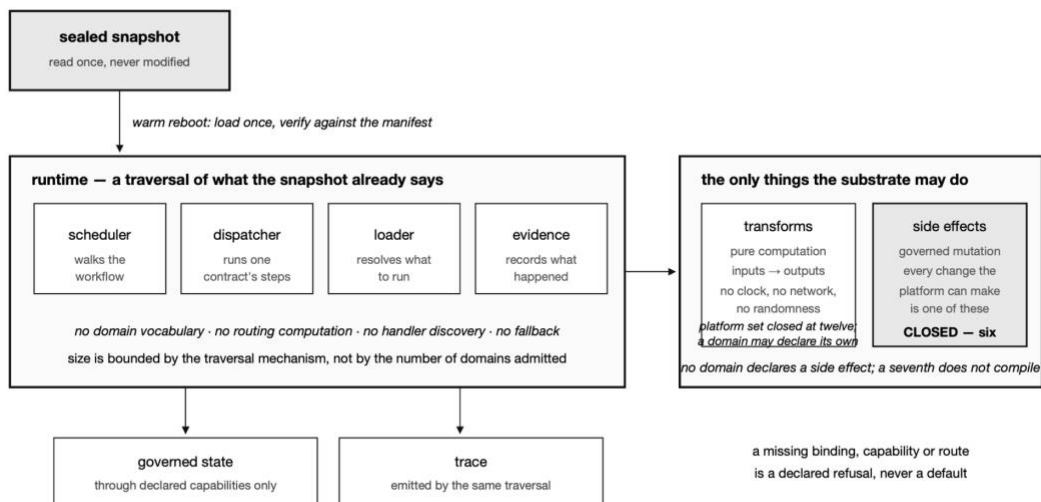
Both surfaces are closed by compiler assertion, and the asymmetry between them is not open versus closed but *who may declare one*. **Mutation is platform-only.** Every governed change to the world goes through one of six side effects named by `INVARIANT_CS_SURFACE_CLOSED_V1` [11]; no domain in the composition declares a side effect of its own, and a seventh does not compile. **Computation may be domain-declared.** The platform transform surface is likewise closed — twelve members, named by `INVARIANT_CT_SURFACE_CLOSED_V1` — but a domain may declare transforms of its own, namespaced to itself and outside that platform surface. Both statements are Enforced.

The consequence is that the full set of effects any composition can produce on anything outside itself is read from six declarations rather than inferred from a codebase.

That closure bounds mutation, and mutation only. It is worth being exact about what it does not bound, because the inference is easy to overstate. A pure transform adds no side effect and can still encode a domain assumption: a validation rule, a control decision, an interpretation of a field. Domain-declared transforms are precisely where such computation is expected to live, and

Section 5.2 places them there deliberately — namespaced to their domain, carried with it, outside the substrate. What the closed platform surface secures is that they cannot accumulate *in the shared surface*: a domain's computation stays addressable to that domain, because the platform list is fixed and a domain cannot add itself to it.

So the closed platform surface does not make domain-specific behavior exhaustively examinable everywhere. It makes the substrate's *effect surface* finite, which is a narrower and checkable thing: whatever a composition does to the world, it does through one of six declared mutations, and each is bound to a store and a consumer the snapshot names. Whether domain-specific behavior has entered the substrate itself is a separate question, answered by inspecting (a) rather than by counting effects. Section 6 reports both.



5.5 A worked admission: actor registration

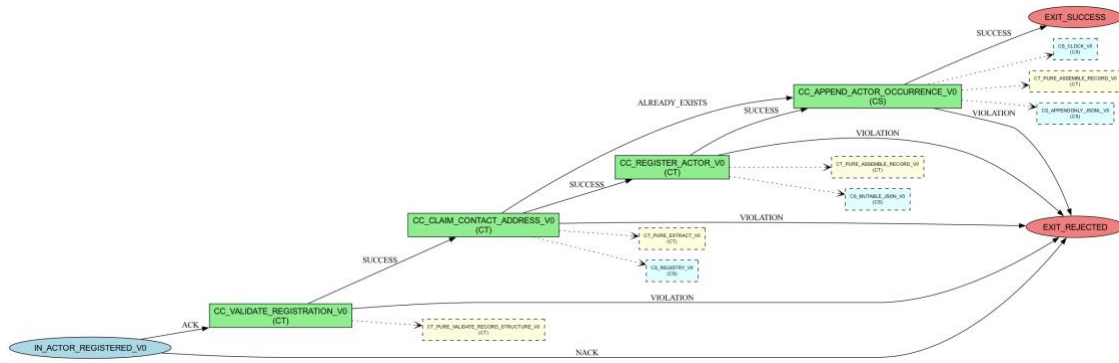
The anatomy and the runtime are easier to check against one workflow than in the abstract. This section takes the identity subdomain of the decentralized identity domain — one of the four

admitted in Section 4.1 — and follows a single workflow: admitting a person as an unverified actor.

The domain author declared nine artifacts for this workflow and wrote no code for it: every capability it reaches is a shared platform one. That is a property of this workflow rather than of the domain, which does declare a transform of its own for deriving a wallet address — a case Section 5.2 counts as domain-carried implementation.

Kind	Artifact	What it fixes
AC	actor context	who the workflow acts for
TI	ingress intent	what may enter at the transport boundary
IN	intent	the admitted request the workflow starts from
WF	workflow	the governed sequence, node by node
CC	capability contract (four of them)	the permitted operation at each step
CT / CS	capability transforms and side effects	what realizes each contract
EV	event	what success announces
RB	runtime binding	which implementation realizes each capability

Figure 4 is the workflow as the composition holds it. It is not drawn for this paper: the assembler renders it from the compiled graph, and it is available for every workflow in every admitted domain. That it can be rendered at all is the point of Section 5.3's indexes — the composition answers the question *what does this workflow do* by projection rather than by reading source.



What the graph fixes. Four capability contracts run in sequence, and each binds to a declared capability rather than to code the workflow supplies.

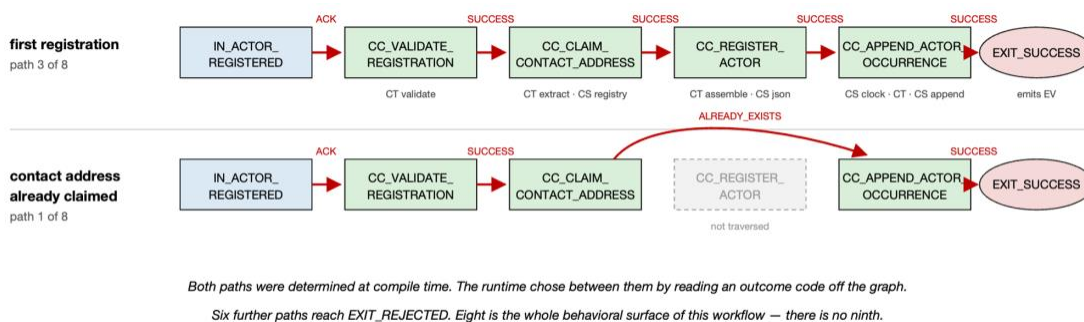
- `CC_VALIDATE_REGISTRATION` binds a pure transform that checks the submitted record against a declared structure.
- `CC_CLAIM_CONTACT_ADDRESS` extracts the contact address and claims it in a registry.
- `CC_REGISTER_ACTOR` assembles the actor record and writes it.
- `CC_APPEND_ACTOR_OCCURRENCE` timestamps the event, assembles an occurrence record and appends it to a stream.

Four of the six side effects appear here — registry, mutable store, append-only stream and clock — and they are named in the artifact rather than discovered at run time. Everything this workflow can do to anything outside itself is in that list. Reaching the world any other way is not a run-time error; it is an artifact that does not compile.

On success the exit emits an event declaring that an unverified actor now exists. The event is declared, so a consumer of it depends on the artifact rather than on the workflow's implementation.

The behavioral surface is enumerable. The compiled graph carries eight execution paths, and eight is all there are. Two reach EXIT_SUCCESS; six reach EXIT_REJECTED. Refusal is not an exceptional case bolted onto a happy path — it is most of the declared surface, and it is countable before anything runs.

Figure 5 traces two of the eight.



The lower path is notable. That a repeat registration of a known contact address should be recorded rather than refused is a domain decision, and a contestable one. In a conventional arrangement it would live inside a service method, discoverable only by reading that method. Here it is an edge in a declared graph: visible in a rendering, reachable by an index query, and counted among the eight.

What is compiled and what arrives at execution. Saying behavior is determined before execution invites an obvious objection: this workflow reads a clock, writes a store and returns different answers on different days, so something is plainly being decided while it runs. The objection is answered by separating two things the word *decide* conflates.

Three things are fixed in the graph and cannot change at execution: which node follows the intent, which capability each node is bound to, and which edge each outcome code selects. Two

things are supplied at execution: the values entering the workflow, and the outcome code a capability returns after computing on them. The runtime never computes an outcome code and never chooses an edge for a code the graph does not carry.

The first path of Figure 5, traced against one submitted record:

Step	Node	Supplied			Effect	
		at execution	Capability invoked	Code returned	Edge taken	outside the workflow
1	IN_ACTOR_REGISTERED	the actor record and a registration schema	—	ACK	to validation	none
2	CC_VALIDATE_REGISTRATION	record, schema	pure structural validation	SUCCESS	to the claim step	none
3	CC_CLAIM_CONTACT_ADDRESS	record, address path, address type	pure extraction, then a registry claim	SUCCESS	to registration	the address is claimed in a registry
4	CC_REGISTER_ACTOR	the extracted address, actor fields	pure record assembly, then a	SUCCESS	to the occurrence step	the actor record is written

Step	Node	Supplied			Effect	
		at execution	Capability invoked	Code returned	Edge taken	outside the workflow
			mutable write			
5	CC_APPEND_ACTOR_OCCURRENCE	occurrence fields, stream id, the claimed address	clock, pure assembly, then an append	SUCCESS	to the success exit	a timestamped occurrence is appended
6	EXIT_SUCCESS	—	—	—	—	the declared event is emitted

Read the *Code returned* column against the *Edge taken* column. Every code was produced by a capability computing on values that arrived at execution; every edge was selected by looking that code up in the compiled graph. Had step 3 returned `ALREADY_EXISTS` instead, the edge to `CC_APPEND_ACTOR_OCCURRENCE` would have been taken — the lower path of Figure 5 — and step 4 would never have run. That alternative was not computed when it happened. It was compiled, months earlier, when the workflow was admitted.

The *Effect outside the workflow* column is the same argument from the other side. Four effects occur, each through one of the six declared side effects, each bound by the runtime binding artifact to a named store. Nothing in the column was decided at execution either: which store, through which side effect, under which binding, are all compiled. What execution supplied was the content written, never the permission to write it.

What the runtime contributed. Nothing in the preceding three paragraphs. The substrate traversing this workflow holds no notion of an actor, a contact address or a registration. It read an intent, looked up an outgoing edge, invoked a bound capability, read the outcome code the capability returned, and followed the edge labelled with that code. The branch at `CC_CLAIM_CONTACT_ADDRESS` was determined before execution; runtime execution only selected the edge corresponding to the returned outcome code.

This is the concrete form of C2. Section 6 measures it across the composition; here it can simply be read off a figure.

5.6 Inspection: the method by which the rest of this paper is checked

Figure 4 was not drawn, and Section 5.5's claim that the workflow carries exactly eight paths was not counted by hand. Both came from a read-only instrument that the composition carries alongside the runtime, and because Section 6's measurements are taken the same way, the instrument belongs in the paper's method rather than in its background.

The inspector. A snapshot inspector reads an assembled snapshot and answers queries over it. It never mutates and never executes. It is a peer of the runtime rather than a part of it: it consumes the sealed artifact by the same read-only path Figure 2 shows, which is why using it to measure the substrate does not disturb what is being measured.

Its properties as an instrument matter more here than its feature list.

- **Its question set comes from the snapshot.** The catalog of operations it can answer is read from artifacts inside the composition, not from a list held privately in the tool. A client's menu is what that snapshot offers. There is no second list to drift from the first.

- **It distinguishes two failure modes.** A well-formed question the snapshot cannot answer as posed returns a negative result; a question no snapshot could ever answer is rejected outright. A measurement that comes back empty is therefore distinguishable from a measurement that was never well posed — the distinction Section 6 depends on when it reports distributions that include zeros.
- **Every check reports what it examined.** Validation returns the examined set alongside the verdict, so a passing check with an empty examined set is visible as such rather than reading as a pass.

The operations Section 6 uses are the artifact catalog and its index membership, the transitive reference closure of an artifact, the consumers of a declared store, and the published execution graph of a workflow — the last of which is what Figure 4 renders.

Two instruments, one discipline. The rendering and the inspector are the same method in two presentations: both derive what they show from the sealed artifact, and neither computes a relationship the composition does not declare. A rendered graph is a projection of declared nodes and edges; a reference closure is a traversal of declared references. Neither is a view a tool author composed.

What they are used for. Two uses are worth naming, because they are the practical reason the architecture carries inspection as a first-class component rather than as tooling.

The first is transformation. Deriving a successor baseline requires facts about the predecessor — what exists, what a proposed change would touch, which workflows reach a given store. Those facts are obtained by querying the sealed predecessor rather than by reading the repository that produced it, so the analysis is performed against what was actually admitted.

The second is finding defects and gaps in declared business behavior before anything runs. Because a workflow's paths are enumerable and its references are traversable, a class of questions that is ordinarily answered by testing becomes answerable by interrogation: whether a declared store has any consumer, whether an artifact is reachable from any workflow, whether a workflow has any path to a successful exit, what the full consumer closure of a capability is. A gap in declared behavior shows up as a missing edge or an empty closure — that is, as an absence in an artifact rather than as a case nobody happened to test.

This is a claim about what the instrument makes visible, not a claim that defects were found. The measurements are in Section 6.

5.7 The admission experiment

Admitting a domain is a transition between sealed states: snapshot n to snapshot $n+1$, where the predecessor is named, the successor derived from it, and the predecessor not edited in place. Stating that transition as a protocol rather than narrating it is what allows C1 and C2 to be measured rather than argued.

Given a snapshot S_n :

1. Admit domain D .
2. Construct S_{n+1} .
3. Compare the identity of the executable substrate, part (a), before and after.
4. Record the capability implementations, part (c), that the admission added, and which domain each is namespaced to.
5. Compare artifacts outside D 's declared paths.

6. Execute S_{n+1} .
7. Retain S_n as the named predecessor.

Steps 3 to 5 are the measurements, and each maps to a condition or to a boundary the conditions depend on.

Step 3 tests C1: if admitting D required the traversal engine to change, its identity differs and the condition fails for that admission.

Step 4 exists because step 3 cannot see it. Part (c) sits outside both identities, so an admission may add implementation code while leaving the substrate byte-identical. Reporting what was added, and where it is namespaced, is what keeps step 3 from being read as a stronger result than it is. An admission that placed a domain-specific implementation alongside the substrate rather than alongside its domain would satisfy step 3 and still refute C2.

Step 5 tests C2 from the artifact side: if construction altered material outside D 's declared paths, then admitting D reached beyond what D declared.

Step 6 matters because an unexecuted snapshot proves nothing about the substrate's sufficiency. A composition that seals but does not run would satisfy every comparison trivially. Step 7 matters because retaining the predecessor is what makes the comparison possible at all; a transition that overwrites its predecessor leaves nothing to compare against.

Section 5.2 settled what counts as the runtime for step 3 and what is treated as an admission artifact rather than a change to the substrate. Section 6 applies those definitions and reports the numbers.

5.8 An architectural consequence not evaluated here

One property of the snapshot is worth stating because it makes a further question well posed, not because this paper answers it.

Alongside the declarations, the snapshot carries a **tokenized projection** of the same graphs. In it the fully qualified names are replaced by integer addresses: adjacency becomes a map from integer to a list of integers, admission becomes a table keyed by the same integers, and the projection's own metadata records that identity by name is no longer required to execute it. For the identity domain examined above, the whole tokenized projection — topology, dispatch, handler table and metadata — is roughly 67 KB covering 51 nodes and 94 edges, against some 296 KB of canonical declarations it was derived from.

Once names are resolved during construction, execution can traverse integer-addressed tables rather than perform name discovery or dynamic behavioral dispatch. This raises the possibility of implementing the traversal substrate in forms other than a general-purpose software runtime, including firmware or fixed logic, with the governed behavior remaining entirely in the artifact the engine is handed. Substrate independence would then hold across a boundary this paper has not examined: not merely one runtime carrying many domains, but one *governed artifact* carried by substrates of materially different kinds.

The present study does not evaluate such implementations, and makes no performance or cost claim about them. It establishes only that the governed behavior can be represented as a sealed structure whose traversal requires no domain knowledge.

6. Measurements

Substrate identity means the content identity of part (a) of §5.2. Changes to the sealed snapshot and to its declared external bindings are admission artifacts rather than modifications to the substrate. Deployment configuration, environment variables, schemas, secrets and external services are part (e) and carry no behavior. The measurements follow from steps 3 to 5 of the admission protocol in §5.7 and are reported against those five parts.

The composition of record is the working composition, snapshot identity 41adfd87.... Its sealed predecessor, deposited and citable, carries identity d92b447f... [12]. Every figure below was taken by inspection of those artifacts and of the version history of the repositories that produced them.

6.1 Measurement 1 — admission independence

The record contains three admissions, covering the four domains. The substrate's identity is the content identity of the runtime package at each.

Admitted	Date	Runtime tree	Substrate	
			changed	Classification
AI governance and licensing; conformance workload	2026-07-	34715db9be	yes	contract
	30			boundary
Library catalog	2026-08-	daf5e358a6	yes	contract
	06			boundary
Identity and wallet	2026-08-	671a19c90e	yes	contract
	11			boundary

Two forms of independence were observed. The executable substrate was not byte-identical across the three recorded admissions, so the strongest form of C1 was not demonstrated. Every observed change was nevertheless located at the shared contract boundary by which the substrate consumes what a snapshot supplies, and none introduced behavior specific to the domain being admitted.

The record therefore supports C1' for these admissions: domain growth did not require domain-specific modification of the executable substrate. That distinction matters because substrate evolution is not necessarily domain coupling. A shared runtime may evolve to consume a new version of a common artifact contract while remaining neutral to the domains that contract carries.

Each change is inspectable, and the reader can check the classification rather than accept it.

- At the first admission the changes were to module loading and to the query surface's internal helpers: fourteen lines inserted, fourteen removed, across boot, loader and three helper modules.
- At the second, the loader began retaining the snapshot root it was loaded from, so that a capability observing a composition binds to the one it is executing inside rather than to a snapshot named by its caller. Sixty-one lines inserted across boot, command line, dispatcher and loader.
- At the third, five lines in the dispatcher: the compiler emits a step's name under one key and the runtime read only the older spelling, so every reference of the form `$.results.<step>.<field>` silently resolved to nothing. A binding the grammar accepted, the compiler rendered, and the runtime dropped.

None of the three mentions a domain. Each repairs a mismatch between what a snapshot supplies and what the substrate expects of any snapshot, which is why the localization claim is the defensible one.

This is C1' of §1.2, and the paper uses that name from here on in place of *unmodified runtime*, which would overstate what this measurement shows.

6.2 Measurement 2 — the implementation inventory

Capability implementations are part (c) of §5.2, outside both identities. The composition loads 61 of them.

Carried with	Modules	Namespaced to
The governance surface	39	the platform
AI governance and licensing	6	its own domain
Library catalog	7	its own domain
Identity and wallet	4	its own domain
Conformance workload	5	its own domain

Every domain-specific implementation is namespaced to the domain that declares it and lives in that domain's repository. None is carried with the substrate. This is the measurement Section 5.7 added step 4 for, and it is the one that could most easily have come out otherwise: a domain-specific module placed with the platform would satisfy Measurement 1 and refute C2, and it would not be visible in any identity comparison.

6.3 Measurement 3 — domain isolation in the composition

No artifact of any admitted domain references any other admitted domain. Across all artifacts of the four domains, cross-domain references number zero.

What this shows is bounded, and the bound matters. It is a property of the composition as it now stands, not a distribution over admissions. The per-admission spillover measurement the protocol calls for — artifacts outside the admitted domain's declared paths that changed during construction of the successor — **is not reported here**, because the intermediate snapshots were not retained. Only the sealed predecessor and the working composition exist as artifacts, so the construction deltas between successive admissions cannot be recovered after the fact. Retaining each successor is a change to the build procedure, not an analysis that can be performed on what was kept.

This limits the conclusion the final composition can support. The present evidence establishes final-state isolation, but not the absence of intermediate spillover during each admission.

6.4 Measurement 4 — domain vocabulary in the substrate

The substrate is 21 modules and 4,372 lines, as deposited [12]. It was searched for the vocabulary of every admitted domain.

Term	Files	Disposition
actor	8	platform vocabulary — the artifact kind that binds authority to a workflow, not a business noun
wallet	1	two occurrences, both in comments illustrating an identifier format

Term	Files	Disposition
agent	1	one occurrence, in a comment illustrating a node key
book, library, catalog, licence, collatz, ISBN, borrower, custodian	0	absent

Vocabulary search is informative but not sufficient. Three occurrences of domain-associated vocabulary are present in the substrate. All three are illustrative text: two are comments, and the third is a command-line help string naming an example workflow — a workflow since renamed, whose staleness went unnoticed precisely because nothing executes it. None participates in an execution decision, and no domain term occurs in a routing decision, dispatch key, handler selection or branch condition.

The stronger inspection therefore asks whether domain vocabulary, routing tables, branch conditions, handler keys or dispatch entries affect execution. No such domain-specific decision was found. The relevant C2 result is consequently not *no domain words occur*, but **no domain-specific execution decision was found in the substrate**. Reported the other way, the measurement that could have refuted C2 was a business noun on an execution path, and the count of those is zero.

What that stronger inspection covered. The substrate's branch conditions, dispatch tables, loader, capability registry and scheduler were also read for any decision taken on the identity of an admitted domain, and none was found: routing reads the compiled graph, dispatch is keyed by node address rather than by name, and the loader resolves whatever a binding names without inspecting what it names. The one place a domain string reaches a decision is the command line, where the user names the workflow to run.

This remains an inspection result rather than a proof against arbitrary encodings of domain behavior. A domain-specific rule can be encoded without a domain's vocabulary — a magic number, a field ordering, a special case on a structural shape only one domain produces. The strongest statement the evidence supports is that no domain-specific decision was found by an inspection that would have found the obvious forms of one.

6.5 Snapshot anatomy

Component counts and sizes for the composition of record, against the classification in §5.3. The file count below includes the manifest, which names the constituents and is therefore not itself one of the 595 constituents §6.6 compares.

Component	Files	Size
canonical declarations	439	2,918 KB
behavior logic	62	5,558 KB
kind index	1	2,987 KB
admission evidence	35	1,955 KB
tokenized projections	28	917 KB
artifact index	1	172 KB
vocabulary	21	72 KB
store index	1	29 KB
trust	7	3 KB
conformance record	1	3 KB
Total	596	14,614 KB

Executable code in the snapshot: none. The behavior logic component is the largest because it carries the rendered projections as well as the graphs, which is what made Figure 4 available without drawing it.

6.6 Reproduction

Two clean rebuilds were performed, each deleting every generated snapshot first. They produced identical snapshot identities, `f9138dec....`. The build is reproducible within one environment, in the sense the reproducible-builds literature gives the term [16].

Compared against the composition as it stood three weeks earlier, identity `41adfd87...`, the result divides sharply. Of 595 constituents, **550 are byte-identical and 45 differ**. Every differing constituent is a rendered image: 31 workflow projection diagrams and 14 evidence views. Every declaration, every execution graph, every tokenized projection, the whole vocabulary and the whole trust component reproduced exactly.

Constituent class	Count	Reproduced
Canonical declarations	439	yes
Execution graphs	31	yes
Tokenized projections	28	yes
Vocabulary	21	yes
Evidence records (JSON)	21	yes
Indexes, trust, conformance	10	yes
Rendered diagrams (PNG)	45	no

The result separates governed content from generated presentation material. The canonical declarations, execution graphs, tokenized projections, vocabulary, evidence records, indexes,

trust component and conformance record reproduced byte-for-byte. The only differences were 45 rendered images.

The governed content was therefore reproducible within the environment, while the current snapshot identity was not environment-independent, because that identity includes renderer-dependent image bytes. The boundary is in what identity covers rather than in what the compiler produces.

That makes the repair precise. Either rendered views are generated deterministically across environments, or they are excluded from the identity of governed content. Neither was done for this paper, and the measurement is reported as it came out.

6.7 Refusal, demonstrated

A refusal case was constructed and run. One capability contract in the identity domain was edited to bind a transform belonging to the library catalog domain — undeclared coupling between two admitted domains, the case §6.3 found zero instances of.

The compiler refused at its second stage, before any artifact was produced.

S2_CANONICALIZE failed with 1 error(s):

[ERROR] E104_INVALID_FQDN: Dangling reference:

blockchain::CC_CLAIM_CONTACT_ADDRESS_V0 → book_library_mgmt::CT_PURE_GROUP
_RECORDS_V0

(target not found in graph or imported surface)

Three properties of the outcome matter more than the refusal itself.

No weaker artifact was produced. The build summary reports one structure failed and none succeeded. There is no partial snapshot, no snapshot with the reference dropped, and no snapshot with the contract omitted. 130 candidate artifacts were proposed and none was admitted.

The refusal is an artifact. The compiler wrote a construction determination recording consequence: REFUSED, the stage that refused, the error code, the offending artifact, the reference that could not be resolved, and the governance closure in force — 80 members under a named closure hash. A refusal leaves a record of the same kind an admission does.

The reason is the namespace closure, not a coupling rule. The determination lists the 27 namespaces the identity domain is authorized to reference. The catalog domain is not among them, so the reference did not resolve. Cross-domain coupling is not refused by a rule that names coupling; it is unreachable because the closure never admitted the namespace. That is why §6.3 could find zero cross-domain references without that being a coincidence — but it also means the zero is a property of the mechanism, and Section 7 records it as such.

The edit was reverted and the composition rebuilt.

6.8 The contribution, stated once

This study provides evidence for a domain-neutral execution substrate within the evaluated scope.

Four heterogeneous domains were admitted and executed by one traversal engine. Their domain-specific implementations remained outside that engine and were namespaced to the domains that declared them. No domain-specific routing or execution decision was found in the substrate. An attempted cross-domain reference was refused during construction, before any artifact was

produced. Together these observations show that one execution mechanism carried materially different governed domains without acquiring execution behavior particular to any of them.

The study also fixes three boundaries of that result.

- The executable substrate changed at the shared contract boundary during the recorded admissions, so literal C1 was not demonstrated. C1' was.
- Per-admission construction spillover was not measured, because intermediate snapshots were not retained. The isolation evidence is a property of the final composition.
- Capability bindings do not fix the content identity of externally loaded implementation code, and snapshot identity includes renderer-dependent presentation artifacts, so it does not reproduce across environments.

The resulting claim is bounded accordingly. The realization demonstrates C1' and supports C2 for the four domains exercised. It does not establish literal substrate immutability, complete implementation provenance, or general substrate independence across the broader space of business software.

This is an architectural case study with four heterogeneous domains and targeted falsification tests, and it does not require PGC to be cheaper, faster, safer or more reliable than a conventional arrangement.

7. Scope and limitations

7.1 What the study establishes

Within the evaluated composition, four heterogeneous domains were admitted and executed by one traversal engine, and the evidence for that is measured rather than asserted.

- **C1' holds for the recorded admissions.** Every substrate change sat at the contract by which the substrate consumes any snapshot, and none was particular to the domain being admitted (§6.1).
- **Domain-specific implementations sit with their domains.** All 22 domain implementations are namespaced to the domain that declares them; none is carried with the substrate (§6.2).
- **No domain-specific execution decision was found in the substrate.** No routing table, branch condition, handler key or dispatch entry is keyed to a domain (§6.4).
- **The final composition contains no cross-domain reference,** and an injected one was refused before any artifact was produced (§6.3, §6.7).
- **Governed content is reproducible within an environment.** Two clean rebuilds produced identical identities, and every declaration, graph, projection and index reproduced byte-for-byte (§6.6).

7.2 What remains untested

The substrate was not byte-invariant. Section 6.1 reports a change to the executable substrate at each admission in the record. The paper does not claim an unmodified runtime in the literal sense, only that no change was particular to an admitted domain. A reader who requires the literal reading should treat C1 as undemonstrated here.

Construction spillover was not measured. The per-admission comparison of artifacts outside a domain's declared paths could not be taken, because intermediate snapshots were not retained. The isolation evidence in Section 6.3 is a property of the final composition, not a distribution over admissions.

Zero cross-domain coupling is partly a property of the mechanism. Section 6.7 shows the namespace closure makes such a reference unresolvable rather than merely disallowed. The zero in Section 6.3 is evidence that the mechanism behaves as specified, not an independent finding about how domains were authored.

7.3 Implementation boundaries

Externally loaded code has no provenance. A binding names a module and a callable and records no digest, version or signature, so none of the attestation the supply-chain literature provides is in force [17, 18, 19]. A substituted module satisfies the binding without changing any identity the paper measures, so the identities reported in Section 6 cover behavioral determination and not the code that realizes it.

Snapshot identity is not reproducible across environments. Section 6.6 reports 45 of 595 constituents differing on rebuild, all of them rendered images. The governed material reproduced exactly; the identity that names it did not.

Domain vocabulary is present in the substrate. Three occurrences, all illustrative, none on an execution path. The narrower claim in Section 6.4 is what the evidence supports.

Each of these is a defect with a known repair — content-pinned bindings, deterministic rendering or an identity that excludes rendered views, and removal of stale text — rather than a limit of the architecture.

7.4 Generalization limits

One realization, one practitioner-author who also designed the architecture, no matched conventional baseline. No claim about cost, delivery time, staffing or defect rates. The property

is demonstrated for the domains exercised and bounded by the table in §4.1: domains requiring external service interaction, transactional or distributed stores, or long-running and time-driven workflows remain untested.

Transformation semantics [6], conformance profiles [11], the lifecycle in which admission sits [10], and the architectural derivation of the inversion itself [8] are each developed elsewhere in the series. This paper is about the snapshot and the runtime that consumes it.

8. Related work

This section applies the criterion of §1.1 to the substrate classes the paper has invoked, and keeps C1 and C2 apart for each. It is comparative positioning rather than the contribution, and it is deliberately short: the paper's centre of gravity is property, mechanism, artifact, measurement, and a related-work section grown into a survey would put it outside the journal's scope.

Recall the test. A substrate contributes domain-specific behavior when, for some admitted artifact, the artifact does not determine a domain-visible outcome and the substrate resolves it.

Substrate class	Unit admitted	C1	C2	What the substrate determines
Operating system	a process	yes	yes	scheduling and resource arbitration — mechanism
Virtual machine	bytecode	yes	yes	class resolution, compilation strategy — mechanism
Database engine	a query	yes	yes	an execution plan, chosen for cost — mechanism
Container orchestrator	a workload	yes	no	restart, backoff, eviction, rollout order

Substrate class	Unit admitted	C1	C2	What the substrate determines
Workflow engine	a process definition	yes	no	gateway expressions, script task results, handler binding
Policy engine	a policy	yes	no	the decision itself, at request time
This realization	a sealed domain artifact	C1'	supported (§6)	edge selection by lookup; no outcome value

Virtual machines are the closest working precedent, and they carry the same weakness. A virtual machine admits new bytecode without change, and its run-time decisions — resolving a class, choosing whether to compile a method — alter how a program runs rather than what it computes. That is the separation this paper asks for at the domain layer.

It is worth noting what a virtual machine does *not* fix, because this paper inherits it exactly. A class is named on a path and resolved at load time; a different class presented under the same name satisfies the reference. Section 5.2 reports the same property for capability modules, and the provenance gap recorded in §7 is the domain-layer form of a problem the platform layer has lived with for thirty years. That is context, not an excuse: it locates the gap as a known class of problem with known repairs rather than as something peculiar to this architecture.

Container orchestration satisfies C1 cleanly and C2 not at all. A declarative workload is admitted unmodified [1], and the controller then chooses the path from observed state to declared state. Restart policy, backoff schedule, eviction order and rollout sequencing are outcomes the workload's author did not determine, and they are domain-visible: whether a job is retried or evicted changes what the system does, not merely how fast. Admission and mutating webhooks move further determination into the substrate [20], which is why §1.1 marks the grouping as

characteristic rather than categorical — an installation without them sits closer to C2 than one with them.

Workflow and process engines are the nearest neighbour, and the comparison is the one a reader will want. A BPMN or similar engine [2, 3] admits a process definition without modification, so C1 holds. It then evaluates gateway expressions, executes script tasks and binds handlers at run time. Under the criterion these are determinations by the engine: the artifact carries an expression, and the engine's evaluator decides what that expression yields. The definition constrains the outcome without determining it.

The difference is narrow and is therefore stated precisely. In the realization studied, an outcome value is computed by a declared capability, and the substrate's only contribution is to look that outcome code up among the edges the compiled graph carries — the trace in §5.5 shows both columns side by side. No expression is evaluated by the engine, because no expression is carried for it to evaluate. Whether that is an advantage depends on what one wants: it buys an enumerable path set and costs the expressiveness of an embedded expression language. This paper measures the first and does not evaluate the second.

Policy engines make the determination explicit and are useful as a contrast. A policy engine [4] exists in order to decide, at request time, what a policy means for a request. That is its purpose rather than a defect, and it satisfies C1 by design. It is cited here only to show that C1 without C2 is a common and often deliberate arrangement, so satisfying C1 alone establishes very little.

Reproducible builds and supply-chain provenance address a different axis, and §6.6 lands in their territory. That literature asks whether an artifact demonstrably came from the sources it

claims — Lamb and Zacchiroli set out the reproducibility case [16], in-toto and comparable frameworks attest the steps of a supply chain [17, 18, 19], and PROV-DM models the provenance record [21]. None of them concerns whether a substrate contributes behavior, so none competes with C1 or C2.

That literature identifies two complementary concerns relevant to this realization. The first is renderer-independent reproducibility: the result in §6.6 is a non-determinism of the kind that literature catalogues, an identity covering generated images that no build pins. The second is implementation provenance: the binding described in §5.2 names a module without pinning its content, which is precisely what supply-chain attestation exists to close. The composition's attestation component is a stub, so neither is addressed here. Both are complementary technologies rather than competing claims, and together they mark what the sealed artifact would need in order to answer *what is running* completely.

Model-driven engineering is the adjacent tradition, and the difference is where the model stops. Model-driven approaches generate an implementation from a model and then run the implementation [22, 23]. The model is a source for construction; it is not the object the substrate executes. In the realization studied the sealed artifact is not a source from which code is generated — it is what the runtime reads at execution, which is why C2 can be checked against the substrate rather than against the generator.

What is not claimed as novel. Sealing a compiled artifact, separating declaration from execution, and refusing rather than repairing are each long-established. The paper's contribution is not a new mechanism but the application of an explicit two-condition test at the business-domain layer, and a measured report of where a realization meets it and where it does not.

9. Conclusion

This study asked whether governed business domains can be admitted as artifacts without placing their behavior into the substrate that executes them.

Within the evaluated scope, the answer is yes for the domain-neutrality part of that question. Four heterogeneous domains were admitted and executed by one traversal substrate. Their domain-specific implementations remained outside the substrate and were namespaced to the domains that declared them. No domain-specific routing, dispatch or execution decision was found in the substrate. An attempted cross-domain reference was refused during construction, before an executable artifact was produced.

The admission history also produced a useful distinction between two forms of substrate independence. The executable substrate changed during the recorded admissions, so literal byte-level C1 was not demonstrated. The observed changes were confined to shared contract boundaries and introduced no behavior for the domains being admitted. C1' — admission without domain-specific substrate modification — was therefore demonstrated for those admissions.

The study also identifies boundaries that remain open. Intermediate snapshots were not retained, so per-admission construction spillover could not be measured. Capability bindings identify implementations by module and callable but do not fix their content identity. Governed content reproduced byte-for-byte within the tested environment, while renderer-dependent presentation artifacts prevented the complete snapshot identity from reproducing across environments.

These results support a bounded architectural claim. A business domain can be represented as governed structure that a shared execution substrate consumes without learning the domain itself.

Domain growth can therefore be separated from growth of domain-specific execution logic in the substrate. The present realization does not establish this property for all business software, nor does it establish implementation provenance or environment-independent snapshot identity.

What the result adds is that the separation can be measured at all. C1 and C2 give explicit criteria for substrate independence, the snapshot gives the object that crosses the boundary, and the measurements show where domain behavior resides, where the substrate changes, and where the current realization still depends on shared infrastructure. The next question is not whether the architecture can carry another domain. It is how far the same separation extends — across richer domains, stronger implementation identity, and independently reproduced substrates.

Declaration of generative AI use

During the preparation of this work the author used Claude/Opus (Anthropic) in the four supporting capacities listed below. After using this tool the author reviewed, revised and edited the content as needed, and takes full responsibility for the content of the publication.

The author is the sole author. The author conceived and wrote the research content — the architectural argument, the two conditions, the measurements, the claims made and the limits placed on them. No generative AI tool is an author, and none could be: a tool cannot hold the accountability that authorship entails. The tool generated no research data, performed no derivation and conducted no analysis of its own.

1. **Drafting assistance under the author's direction.** The assistant drafted and restructured prose from the author's outline, argument and decisions. The author reviewed, revised and accepted every passage. No text entered the manuscript without that review.

2. **Rendering Figures 1, 2, 3 and 5.** The author supplied each figure's content: the decomposition, the partitions, the properties and the placement of each property at its point of origin. The assistant produced the drawings and refined their layout. These are conceptual schematics. They present no research data, no experimental results and no data visualization, and no data, result or figure presenting data was generated or altered by any AI tool. Figure 4 is not an AI product: it is rendered from the compiled snapshot by the composition's own inspector, as §5.6 states.
3. **Extracting material from the reference realization.** The observations reported in Section 6 were read from the repositories, build outputs, snapshot inspections and execution traces of the reference realization and transcribed into the manuscript. The implementation, not the manuscript, is the record.
4. **Proofreading and copy-editing.**

A separate disclosure belongs to the reference realization itself, because it bears on Section 6. Parts of that realization were constructed by AI workers performing governed activity under the author's direction, within the architecture this paper describes. The author held scope, admission and promotion authority throughout; the workers held none. That arrangement is not what this paper measures, and the results in Section 6 do not depend on it. A companion study examines it directly [9].

Data and code availability

The realization is open source and the composition of record is deposited and citable. The sealed predecessor examined in Section 6, identity d92b447f..., together with the ten repositories that compose it, is archived at [12]. The measurements in Section 6 were taken with the snapshot

inspector described in §5.6, which is part of that deposit. The specification the composition claims conformance to is [11].

References

- [1] Burns, B., Grant, B., Oppenheimer, D., Brewer, E. and Wilkes, J. (2016) Borg, Omega, and Kubernetes. *ACM Queue*, **14**, 70–93. DOI: [10.1145/2890784](https://doi.org/10.1145/2890784).
- [2] Object Management Group (2011) *Business Process Model and Notation (BPMN), Version 2.0*. <https://www.omg.org/spec/BPMN/>
- [3] van der Aalst, W. M. P. (2013) Business process management: a comprehensive survey. *ISRN Software Engineering*, **2013**, Article 507984. DOI: [10.1155/2013/507984](https://doi.org/10.1155/2013/507984).
- [4] Open Policy Agent (2026) *Open Policy Agent*. Cloud Native Computing Foundation. <https://www.openpolicyagent.org>
- [5] Ganti, B. (2026) *Protocol-Governed Computing: An Architecture for Deterministic Declarative Execution*. Zenodo. DOI: [10.5281/zenodo.21879516](https://doi.org/10.5281/zenodo.21879516).
- [6] Ganti, B. (2026) *Protocol-Governed Computing: An Architecture for Closed-Loop Governed Transformation*. Zenodo. DOI: [10.5281/zenodo.21879948](https://doi.org/10.5281/zenodo.21879948).
- [7] Ganti, B. (2026) *Who Authorizes Software Behavior? Governing the AI-Native SDLC*. Manuscript under review, *IEEE Computer*. Author's copy: <https://omnibachi.org/papers/who-authorizes-software-behavior/>

- [8] Ganti, B. (2026) *Protocol-Governed Computing: Architectural Inversion, Its Consequences, and a Scaling Claim*. Manuscript under review, *Software: Practice and Experience*. Preprint DOI: [10.5281/zenodo.22736531](https://doi.org/10.5281/zenodo.22736531).
- [9] Ganti, B. (2026) *Protocol-Governed Human-AI Software Engineering: Autonomy Without Authority*. Manuscript under review, *Automated Software Engineering*. Preprint DOI: [10.5281/zenodo.22650863](https://doi.org/10.5281/zenodo.22650863).
- [10] Ganti, B. (2026) *Protocol-Governed Computing: A Software Development Lifecycle Architecture for Deterministic Declarative Execution and Governed Transformation*. Manuscript under review, *Journal of Systems and Software*. Preprint DOI: [10.5281/zenodo.22758703](https://doi.org/10.5281/zenodo.22758703).
- [11] Ganti, B. (2026) *Open Protocol-Governed Computing Standard, revision v0*. Zenodo. DOI: [10.5281/zenodo.22150616](https://doi.org/10.5281/zenodo.22150616).
- [12] Ganti, B. (2026) *Protocol-Governed Computing: composition deposit v4*. Zenodo. DOI: [10.5281/zenodo.22714911](https://doi.org/10.5281/zenodo.22714911).
- [13] Morris, K. (2016) *Infrastructure as Code: Managing Servers in the Cloud*. O'Reilly Media. ISBN 978-1-4919-2435-8.
- [14] Dennis, J. B. and Van Horn, E. C. (1966) Programming semantics for multiprogrammed computations. *Communications of the ACM*, **9**, 143–155. DOI: [10.1145/365230.365252](https://doi.org/10.1145/365230.365252).
- [15] Miller, M. S. (2006) *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University.
- [16] Lamb, C. and Zacchiroli, S. (2022) Reproducible builds: increasing the integrity of software supply chains. *IEEE Software*, **39**, 62–70. DOI: [10.1109/MS.2021.3073045](https://doi.org/10.1109/MS.2021.3073045).

- [17] Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R. and Cappelletti, J. (2019) in-toto: providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pp. 1393–1410.
- [18] Newman, Z., Meyers, J. S. and Torres-Arias, S. (2022) Sigstore: software signing for everybody. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 2353–2367. DOI: [10.1145/3548606.3560596](https://doi.org/10.1145/3548606.3560596).
- [19] Open Source Security Foundation (2026) *SLSA: Supply-chain Levels for Software Artifacts*. <https://slsa.dev>
- [20] Kubernetes (2026) *Admission Control in Kubernetes*. The Kubernetes Authors. <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/>
- [21] Moreau, L. and Missier, P. (eds) (2013) *PROV-DM: The PROV Data Model*. W3C Recommendation, 30 April 2013. <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>
- [22] Schmidt, D. C. (2006) Model-driven engineering. *IEEE Computer*, **39**, 25–31. DOI: [10.1109/MC.2006.58](https://doi.org/10.1109/MC.2006.58).
- [23] France, R. and Rumpe, B. (2007) Model-driven development of complex software: a research roadmap. In *Future of Software Engineering*, pp. 37–54. DOI: [10.1109/FOSE.2007.14](https://doi.org/10.1109/FOSE.2007.14).